How can Docker help solve the problem "It works on my machine"?

Handout · Justus Bruegmann· TINF25B5 · 10-26-2026

Short answer

> Docker can solve the “It works on my machine” problem. It is especially helpful for testing environments where a backend or other services need to run alongside your system. Furthermore, it can simplify deploying your application.

The problem

Code that runs on one machine fails on another. Typical causes:

- Different runtime versions (e.g. Node, Python, Java)
- Missing or different libraries and dependencies
- System packages installed on one machine only
- Environment variables and local configuration
- Different operating systems

Containers vs. virtual machines

- A virtual machine runs a complete guest operating system on a hypervisor.
- A container shares the host kernel and only packages the app and its dependencies, so it starts in seconds and is much smaller.
- Isolation comes from Linux namespaces (what a process can see) and cgroups (how much CPU and memory it can use).

Dockerfile example

```
# Base image with a pinned version
FROM node:22-alpine
WORKDIR /app
# Copy dependency files first to use the build cache
COPY package*.json ./
RUN npm ci
# Then copy the source code
COPY . .
EXPOSE 3000
CMD ["npm", "start"]
```

Limits

- CPU architecture: images built for ARM (Apple Silicon) may not run on x86 and vice versa.
- Secrets and local .env files still have to be shared separately.
- Bind mounts can be slow on macOS and Windows.
- Unpinned tags such as latest make builds unreproducible again.

Command cheat sheet

Command	What it does
`docker ps`	Lists all running containers (-a also shows stopped ones)
`docker image ls`	Lists all downloaded images
`docker run <image>`	Creates and starts a container from an image
`docker <start|stop|rm> <container>`	Starts, stops or removes a container by name or hash
`docker build -t <tag> <build_context>`	Builds an image from a Dockerfile
`docker logs <container>`	Prints the logs of a container
`docker compose up`	Starts all services from compose.yml
`docker compose stop`	Stops all services
`docker compose down`	Stops and deletes all services
`docker compose logs`	Prints the logs of all services

Sources

- How does Docker actually work? The hard way (Furkan Türkal, Medium). https://medium.com/@furkan.turkal/how-does-docker-actually-work-the-hard-way-a-technical-deep-diving-c5b8ea2f0422
- Docker lightweight images explained: Alpine vs. Ubuntu 24.04 (Medium). https://cloudbloggeracademy.medium.com/docker-lightweight-images-explained-alpine-vs-ubuntu-24-04-c971c2c24f0c
- Ubuntu Server system requirements. https://ubuntu.com/server/docs/reference/installation/system-requirements/
- namespaces(7), Linux manual page. https://man7.org/linux/man-pages/man7/namespaces.7.html
- cgroups(7), Linux manual page. https://man7.org/linux/man-pages/man7/cgroups.7.html
- Docker Docs: What is a registry? https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-registry/
- Docker Docs: Multi-stage builds. https://docs.docker.com/build/building/multi-stage/
- Docker Docs: Compose. https://docs.docker.com/compose/